



## Python Programs Collection

This repository contains several beginner-to-intermediate level Python programs that solve common data structure and text-processing problems. Each script includes an example input and corresponding output.

### 1. Find Common Elements (without duplicates)

#### Problem:

Find all elements that appear in both lists, ensuring duplicates appear only once.

```
def find_common_elements(list1, list2):  
    common = []  
    for item in list1:  
        if item in list2 and item not in common:  
            common.append(item)  
    return common  
  
# Example  
list1 = [1, 2, 3, 4]  
list2 = [3, 4, 5, 6]  
  
result = find_common_elements(list1, list2)  
print(result) # Output: [3, 4]
```

### 2. Word Frequency Counter

#### Problem:

Given a sentence, count how many times each word appears.

```
def word_frequency(sentence):  
    # Convert to lowercase  
    sentence = sentence.lower()
```

```

# Split into words
words = sentence.split()

# Dictionary to store word counts
freq = {}

for word in words:
    if word not in freq:
        freq[word] = 1
    else:
        freq[word] += 1

return freq

# Example
sentence = "the cat and the hat"
result = word_frequency(sentence)
print(result) # Output: {'the': 2, 'cat': 1, 'and': 1, 'hat': 1}

```

### 3. Transpose a Matrix

#### Problem:

Given a 2D list (matrix), print its transpose.

```

def transpose_matrix(matrix):
    # Use list comprehension to transpose
    transposed = [[row[i] for row in matrix] for i in range(len(matrix[0]))]
    return transposed

# Example
matrix = [[1, 2], [3, 4], [5, 6]]
result = transpose_matrix(matrix)
print(result) # Output: [[1, 3, 5], [2, 4, 6]]

```

## 4. Reverse Words in a Sentence

### Problem:

Write a program that reverses the order of words in a string.

```
def reverse_words(sentence):  
    # Split the sentence into words  
    words = sentence.split()  
  
    # Reverse the list of words  
    words.reverse()  
  
    # Join the reversed list back into a string  
    reversed_sentence = ' '.join(words)  
  
    return reversed_sentence  
  
# Example  
sentence = "hello world python"  
result = reverse_words(sentence)  
print(result) # Output: "python world hello"
```

## 5. Remove Duplicates (Preserve Order)

### Problem:

Remove duplicates from a list without using `set()`, while preserving the original order.

```
def remove_duplicates(lst):  
    unique_list = []  
    for item in lst:  
        if item not in unique_list:  
            unique_list.append(item)  
    return unique_list  
  
# Example  
numbers = [4, 2, 4, 5, 2, 3, 5, 1]  
result = remove_duplicates(numbers)
```

```
print(result) # Output: [4, 2, 5, 3, 1]
```

## 6. Find All Unique Pairs That Add Up to a Target Sum

### Problem:

Given a list of numbers and a target sum, find all unique pairs that add up to that sum.

```
def find_pairs(lst, target):
    pairs = []
    seen = set()

    for num in lst:
        complement = target - num
        if complement in lst and (min(num, complement), max(num, complement)) not in
seen:
            # Avoid duplicates and count each pair only once
            pairs.append((num, complement))
            seen.add((min(num, complement), max(num, complement)))
    return pairs

# Example
numbers = [2, 4, 3, 5, 7, 8, -1]
target = 6
result = find_pairs(numbers, target)
print(result) # Output: [(2, 4), (7, -1)]
```